LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

02

Working with Linux Files and Directories

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 The Linux Filesystem
02 Absolute vs Relative Path
03 Navigating the Filesystem
04 Managing Files
05 Viewing and Editing Files
06 Hidden Files and Extensions
07 Hard Links vs Symbolic Links
08 File Metadata and Inodes
09 Searching Files and Content
10 File Timestamps
11 Archiving and Compressing
12 Interview Q&A
13 Takeaways and Exercises

Ravi Tomar
Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux
github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand how Linux organizes files and directories, how to navigate the filesystem, manage files, search for information and work with file links.

1 Understanding the Linux Filesystem

Linux uses a hierarchical filesystem structure. Unlike Windows, which commonly uses drive letters such as `C:` and `D:`, Linux organizes everything under a single root directory represented by `/`.

All files, directories, mounted storage devices and other filesystem resources are accessible through this hierarchy.

Important Linux Directories

Directory	Purpose
`/`	Root directory; starting point of the filesystem.
`/home`	Contains home directories of normal users.
`/root`	Home directory of the root user.
`/etc`	System-wide configuration files.
`/var`	Variable data such as logs, caches and application data.
`/var/log`	System and application log files.
`/tmp`	Temporary files.
`/usr`	User-space programs, libraries and shared resources.
`/bin`	Essential user commands; commonly linked to `/usr/bin` on modern systems.
`/sbin`	System administration commands; commonly linked to `/usr/sbin` on modern systems.
`/opt`	Optional or third-party application software.
`/boot`	Files required for the boot process, including kernel images.
`/dev`	Device files representing hardware and virtual devices.
`/proc`	Virtual filesystem exposing process and kernel information.
`/sys`	Virtual filesystem exposing device and kernel information.
`/mnt`	Common temporary mount point for filesystems.
`/media`	Common mount location for removable media.

> **IMPORTANT**
>
> `/proc` and `/sys` are virtual filesystems. They do not behave like ordinary directories containing files stored on disk.

Practical Example

If you deploy a Node.js application on a Linux server, you might organize it like this:

```
/opt/myapp/          → Application files
/etc/myapp/          → Application configuration
/var/log/myapp/      → Application logs
/var/lib/myapp/      → Persistent application data
```

This separation makes application management, troubleshooting and backups easier.

> **INTERVIEW QUESTION**
>
> **What is the difference between `/etc`, `/var` and `/opt`?**
>
> - `/etc` stores configuration files.
> - `/var` stores data that changes during system operation, such as logs.
> - `/opt` is commonly used for optional or third-party software installations.

2 Absolute Path vs Relative Path

A path identifies the location of a file or directory. Linux supports two main types of paths.

Absolute Path

An absolute path starts from the root directory `/` and specifies the complete location of a file or directory.

```
/etc/nginx/nginx.conf
/var/log/syslog
/home/ubuntu/application
```

Relative Path

A relative path specifies a location based on the current working directory.

```
cd /var
cd log
```

Here, `log` is a relative path because it is accessed from the current directory `/var`.

Important Path Symbols

Symbol	Meaning
`/`	Root directory
`.`	Current directory
`..`	Parent directory
`~`	Current user's home directory
`-`	Previous working directory when used with `cd -`

INTERVIEW QUESTION

What is the difference between an absolute and relative path?

An absolute path starts from the root directory and provides the complete location. A relative path is interpreted based on the current working directory.

3 Navigating the Linux Filesystem

These are the most commonly used commands when working with Linux servers.

Command	Purpose
`pwd`	Displays the current working directory.
`ls`	Lists files and directories.
`ls -l`	Displays detailed file information.
`ls -a`	Displays hidden files as well.
`ls -lh`	Displays file sizes in human-readable format.
`cd`	Changes the current directory.
`cd ..`	Moves to the parent directory.
`cd ~`	Moves to the user's home directory.
`cd -`	Returns to the previous working directory.
`tree`	Displays directories in a tree structure (if installed).

Practical Examples

```
# Display current directory
pwd

# List files
ls

# Show hidden files
ls -la

# Display detailed information
ls -lh

# Move to application directory
cd /opt/myapp

# Move one level up
cd ..

# Return to home directory
cd ~
```

IMPORTANT

Linux is case-sensitive. `Application`, `application` and `APPLICATION` are treated as different names.

4 Creating, Copying, Moving and Deleting Files

Linux provides commands to manage files and directories directly from the terminal.

Important Commands

Command	Purpose
`touch`	Creates an empty file or updates timestamps.
`mkdir`	Creates a directory.
`mkdir -p`	Creates parent directories if they don't exist.
`cp`	Copies files.
`cp -r`	Copies directories recursively.
`mv`	Moves or renames files and directories.
`rm`	Deletes files.
`rm -r`	Deletes directories recursively.
`rmdir`	Deletes empty directories.
`file`	Identifies the type of a file.
`stat`	Displays detailed file metadata.

Practical Examples

```
# Create a file
touch app.log

# Create a directory
mkdir myapp

# Create nested directories
mkdir -p /opt/myapp/logs/archive

# Copy a file
cp app.log backup.log

# Copy a directory
cp -r myapp myapp-backup

# Rename a file
mv app.log application.log

# Move a file
mv application.log /var/log/

# Delete a file
rm backup.log

# Delete an empty directory
rmdir myapp
```

Using Wildcards

Wildcards let you work with many files at once by matching patterns in filenames.

Wildcard	Meaning	Example
`*`	Matches any number of characters	`rm *.tmp`
`?`	Matches exactly one character	`ls file?.log`
`[ ]`	Matches one character from a set or range	`ls app[1-3].log`

PRODUCTION TIP

Before deleting with a wildcard, run `ls` with the same pattern first to preview exactly which files will be removed.

Important: Be Careful with rm

The `rm` command generally deletes files without moving them to a recycle bin.

```
rm -rf directory/
```

This recursively deletes the directory and its contents without prompting for confirmation in normal circumstances.

> **PRODUCTION WARNING**
>
> Always verify the path before running destructive commands, especially when using variables in deployment and cleanup scripts.

5 Viewing and Editing Files

Linux administrators frequently inspect configuration files, application logs and system information.

Common File Viewing Commands

Command	Purpose
`cat`	Displays file contents.
`less`	Opens a file for interactive reading.
`more`	Displays content page by page.
`head`	Displays the first 10 lines by default.
`tail`	Displays the last 10 lines by default.
`tail -f`	Continuously follows new lines added to a file.
`nl`	Displays file contents with line numbers.

Practical Examples

```
# Display complete file
cat /etc/hostname

# Read large files page by page
less /var/log/syslog

# Display first 20 lines
head -n 20 application.log

# Display last 50 lines
tail -n 50 application.log

# Follow logs in real time
tail -f /var/log/nginx/access.log

# Display line numbers
nl application.conf
```

> **NOTE**
>
> `/var/log/syslog` exists on Ubuntu and Debian. RHEL-based systems use `/var/log/messages` instead.

Editing Files

Two commonly used terminal editors are:

- `nano` – Simple and beginner-friendly.
- `vi` / `vim` – Powerful editors commonly available on Linux servers.

```
# Using nano
nano /etc/hostname

# Using Vim
vim application.conf
```

Important Vim Commands

Command	Purpose
`i`	Enter insert mode.
`Esc`	Return to normal mode.
`:w`	Save changes.
`:q`	Quit.
`:wq`	Save and quit.
`:q!`	Quit without saving.
`/text`	Search for text.

DEVOPS EXAMPLE

When an application fails to start, you may inspect its configuration file using `cat` or `less`, edit the configuration using `vim`, and monitor its logs using `tail -f`.

6 Hidden Files and File Extensions

Hidden Files

In Linux, a filename beginning with a dot (`.`) is treated as hidden by standard directory listing commands.

```
.bashrc
.bash_profile
.gitconfig
.env
.ssh/
```

These files are commonly used for shell configuration, application environment variables, Git configuration and SSH settings.

```
# Display normal files
ls

# Display hidden files
ls -a

# Display detailed information including hidden files
ls -la
```

File Extensions

Unlike Windows, Linux does not rely on file extensions to determine whether a file is executable or what type it is.

```
deploy.sh
application.conf
backup.tar
```

A `.sh` file is commonly a shell script, but its execution depends on permissions, its contents and how it is invoked.

Use the following command to identify file types:

```
file deploy.sh
file application.conf
```

> INTERVIEW QUESTION
>
> **Does Linux require file extensions?**
>
> No. Linux identifies files using their contents and metadata rather than relying on extensions. File extensions are mainly used as naming conventions.

7 Hard Links vs Symbolic Links

Linux supports links that allow multiple paths to reference file data or another file path. There are two important types.

Hard Link

A hard link is another directory entry pointing to the same inode as the original file. Both names refer to the same underlying file data.

```
ln original.txt hardlink.txt
```

Symbolic Link (Soft Link)

A symbolic link is a separate file containing a reference to another file or directory path.

```
ln -s original.txt softlink.txt
```

Difference Between Hard Link and Soft Link

Feature	Hard Link	Symbolic Link
Points to	Same inode	Target path
Inode	Same as original	Different inode
Across filesystems	No	Yes
Directory linking	Normally not allowed	Allowed
If original is deleted	Link data remains accessible	Link becomes broken if target path no longer exists

Practical Example

```
# Create original file
echo "Linux DevOps Guide" > original.txt

# Create hard link
ln original.txt hardlink.txt

# Create symbolic link
ln -s original.txt softlink.txt

# View inode numbers
ls -li
```

INTERVIEW QUESTION

What happens if you delete the original file?

- A hard link continues to access the data because it references the same inode.
- A symbolic link may become broken because it points to the original pathname.

DEVOPS USE CASE

Symbolic links are commonly used to manage application versions. For example:

```
/opt/myapp/releases/v1
/opt/myapp/releases/v2
/opt/myapp/current → /opt/myapp/releases/v2
```

The `current` symbolic link can be updated to point to a new release during deployment.

8 File Metadata and Inodes

What Is an Inode?

An inode is a filesystem data structure that stores metadata about a file, such as:

- File permissions.
- File ownership.
- File size.
- Timestamps.
- References to the file's data blocks.
- Link count.

The filename itself is stored in a directory entry, which maps the name to an inode number.

Check Inode Information

```
# Display inode numbers
ls -i

# Display detailed metadata
stat application.log

# Check inode availability
df -i
```

What Is Inode Exhaustion?

A filesystem has a limited number of inodes. If all available inodes are consumed, the system may be unable to create new files even when disk space is available.

Real-Life Example

An application generates thousands of small temporary files and never deletes them. Eventually:

- Disk space may still be available.
- Inodes may become exhausted.
- The application cannot create additional files.

Check using:

```
df -h
df -i
```

> INTERVIEW QUESTION
>
> **Can a Linux server have free disk space but still fail to create files?**
>
> Yes. This can happen when the filesystem runs out of available inodes.

9 Searching for Files and Content

Searching is an important daily task for DevOps engineers, especially when investigating configuration files and application logs.

Finding Files: find

The `find` command searches for files and directories based on conditions such as name, type, size and modification time.

```
# Find a file by name
find /etc -name "nginx.conf"

# Find all log files
find /var/log -name "*.log"

# Find directories
find /opt -type d

# Find files larger than 100 MB
find /var -type f -size +100M

# Find files modified within the last 24 hours
find /var/log -type f -mtime -1
```

Searching Text: grep

The `grep` command searches for matching text inside files or command output.

```
# Search for an error
grep "ERROR" application.log

# Case-insensitive search
grep -i "error" application.log

# Display matching line numbers
grep -n "failed" application.log

# Search recursively in a directory
grep -r "connection refused" /var/log/

# Show matching lines and surrounding context
grep -C 3 "ERROR" application.log
```

Combining Commands

```
# Find all log files and search for ERROR
find /var/log -type f -name "*.log" -exec grep -nH "ERROR" {} \;
```

This searches matching log files and displays the filename and line number for each match.

> **DEVOPS EXAMPLE**
>
> When a deployment fails, use `grep` to search for errors in application logs and `find` to locate logs across application directories.

10 Understanding File Timestamps

Linux maintains three commonly discussed file timestamps.

Timestamp	Meaning
atime	Last access time.
mtime	Last content modification time.
ctime	Last metadata change time, such as permissions or ownership.

```
# Check timestamps
stat application.log

# Update file timestamps
touch application.log
```

IMPORTANT

ctime is not file creation time. Birth time may also be available depending on the filesystem and system tools.

INTERVIEW QUESTION

What is the difference between mtime and ctime?

mtime represents when file contents were last modified. ctime represents when inode metadata was last changed.

11 Archiving and Compressing Files

DevOps engineers regularly bundle and compress files for backups, deployments and log retention.

tar and gzip

- **tar** combines multiple files and directories into a single archive file.
- **gzip** compresses a single file to reduce its size.

Together they produce the common `.tar.gz` format.

Common tar Options

Option	Purpose
`-c`	Create a new archive.
`-x`	Extract an archive.
`-t`	List the contents of an archive.
`-v`	Show progress (verbose).
`-f`	Specify the archive file name.
`-z`	Compress or decompress using gzip.
`-C`	Extract into a specific directory.

Practical Examples

```
# Create a compressed archive
tar -czvf myapp-backup.tar.gz /opt/myapp

# List archive contents without extracting
tar -tzvf myapp-backup.tar.gz

# Extract in the current directory
tar -xzvf myapp-backup.tar.gz

# Extract into a specific directory
tar -xzvf myapp-backup.tar.gz -C /tmp/restore

# Compress and decompress a single file
gzip application.log
gunzip application.log.gz

# Search inside a compressed log without extracting it
zgrep "ERROR" application.log.1.gz
```

NOTE

When you archive an absolute path such as `/opt/myapp`, tar removes the leading `/` from member names. This is a safety feature, so extracting the archive does not overwrite the original location.

DEVOPS EXAMPLE

Rotated logs are often compressed as `.gz` files. Use `zgrep` or `zcat` to search them during an investigation without extracting them first.

INTERVIEW QUESTION

What is the difference between tar and gzip?

tar bundles multiple files into a single archive but does not compress by default. gzip compresses a single file. Using `tar -z` combines both to create a compressed `.tar.gz` archive.

12 Important Interview Questions and Answers

Q1. What is the root directory in Linux?

The root directory `/` is the starting point of the Linux filesystem hierarchy. All files, directories and mounted filesystems are accessible under it.

Q2. What is the difference between `/root` and `/`?

`/` is the root of the entire filesystem. `/root` is the home directory of the root user.

Q3. What is the difference between `/etc` and `/var`?

`/etc` stores system configuration files, while `/var` stores changing data such as logs, caches and application data.

Q4. How do you find files larger than 1 GB?

```
find / -type f -size +1G 2>/dev/null
```

Q5. How do you monitor a log file continuously?

```
tail -f application.log
```

Q6. What is the difference between `rm` and `rmdir`?

`rm` removes files and can remove directories recursively with `-r`. `rmdir` removes only empty directories.

Q7. What is the difference between a hard link and a symbolic link?

A hard link points to the same inode as the original file. A symbolic link points to another file or directory path.

Q8. What is an inode?

An inode stores file metadata and references to its data blocks. A directory entry associates a filename with an inode.

Q9. Why can a file not be created even when disk space is available?

The filesystem may have exhausted its inodes. Check using `df -i`.

Q10. How do you find and investigate large log files?

```
find /var/log -type f -size +100M
du -sh /var/log/*
```

These commands help identify large files and directories for further investigation.

Q11. How do you create and extract a compressed archive?

```
# Create
tar -czvf backup.tar.gz /opt/myapp

# Extract
tar -xzvf backup.tar.gz
```

13 Chapter Takeaways

After completing this chapter, you should be comfortable with:

- ✓ Understanding the Linux directory hierarchy.
- ✓ Navigating using absolute and relative paths.
- ✓ Creating, copying, moving and deleting files.
- ✓ Using wildcards safely.
- ✓ Viewing and editing configuration files and logs.
- ✓ Understanding hidden files and file extensions.
- ✓ Working with hard links and symbolic links.

- ✓ Understanding inodes and file metadata.
- ✓ Searching files using `find` and text using `grep`.
- ✓ Understanding file timestamps.
- ✓ Archiving and compressing files using `tar` and `gzip`.
- ✓ Managing files safely on production servers.

PRACTICAL EXERCISES

Try these tasks on an Ubuntu or Amazon Linux VM:

1. Create `/opt/devops/app/logs` using a single command.
2. Create a file called `application.log` inside it.
3. Add some sample log entries and display the last five lines.
4. Find all `.log` files under `/var/log`.
5. Search for the word `error` inside a sample log file.
6. Create a symbolic link pointing to your application directory.
7. Display the inode number of a file.
8. Check available disk space and inode availability.
9. Create a compressed archive of `/opt/devops` and extract it into `/tmp/restore`.

END OF CHAPTER 2